ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ И ИНФОРМАТИКИ»

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине “Моделирование”

Выполнил студент Рудцких Владислав Евгеньевич

Ф.И.О.

Группы ИВ-222

Работу принял Ассистент кафедры ВС Уженцева А. В.

подпись

Защищена _____ Оценка

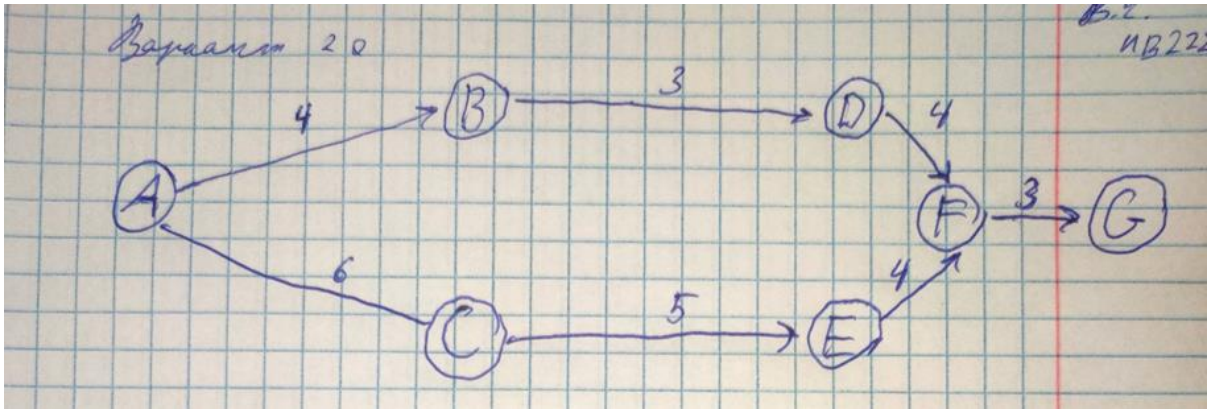
Новосибирск – 2026

Задание

Вариант 20:

Вариант 20

Вершина	Предшественник	Длительность
A	-	5
B	A	4
C	A	6
D	B	3
E	C	5
F	D,E	4
G	F	3



Реализовать построение сетевого графика и расчет критических точек по взвешенному графу событий.

Теория

Шифр - две вершины связанные ребром

t_ij - вес (длительность) ребра

t^РН_ij - самый ранний возможный момент времени, когда данная работа может быть начата при условии, что все предшествующие работы выполнены в строго запланированной последовательности

t^РО_ij - самый ранний из возможных сроков окончания работы

t^ПН_ij - самый поздний срок при котором может быть начата работа без нарушения общего срока работы

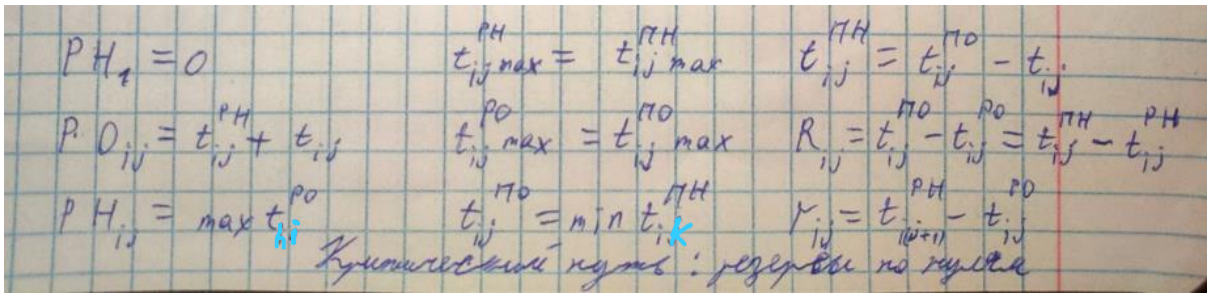
t^ПО_ij - самый поздний срок при котором можно закончить работу не срывая срок проекта

R_ij - максимальное время на которое можно увеличить время работы без нарушения позднего срока конечного события сохраняя продолжительность критического пути

r_ij - количество времени на которое можно увеличить продолжительность работы или перенести ее начало так, чтобы не изменилось ранее начало последующих работ

T_кр - сумма длительностей всех операций критического пути

Сетевой график - графическая модель проекта описывающего логические взаимосвязи и последовательности между операциями



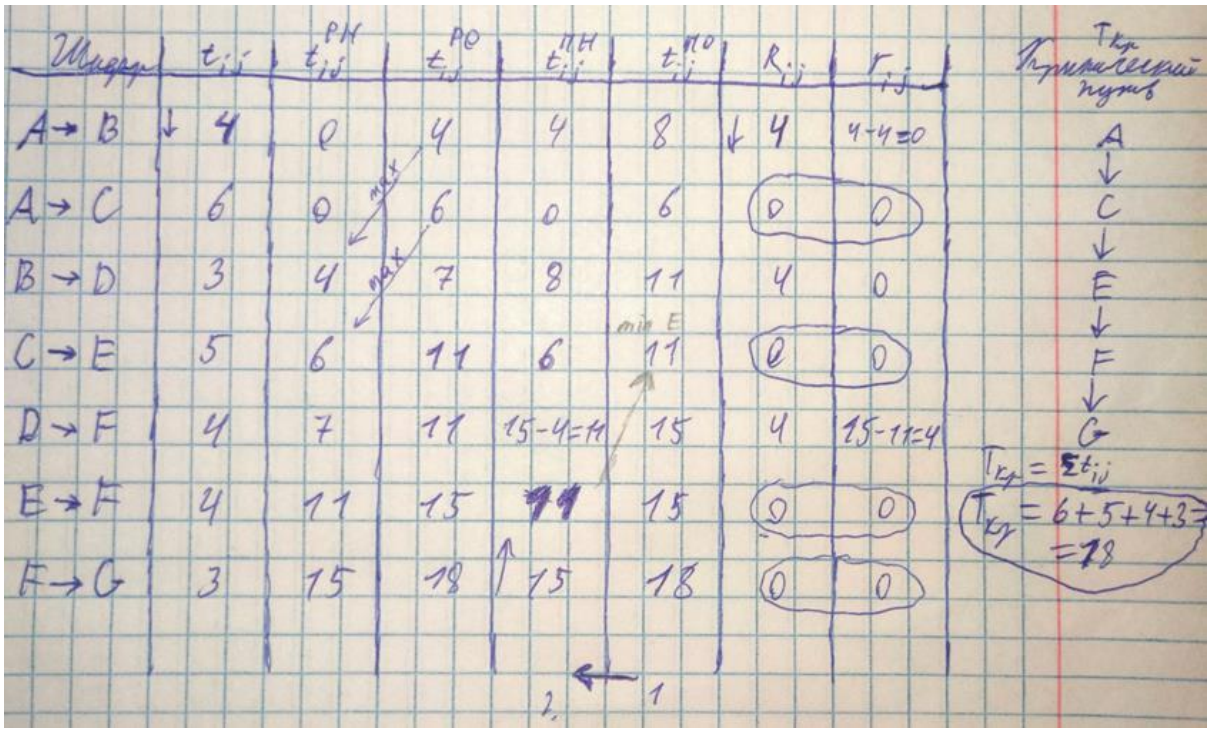
Ход работы

Выполнение вычислений:

1) Построение графа на основе таблицы

2) Построение сетевого графика

3) Нахождение критического пути



Шифр работ	t_{ij}	$t_{ij}^{РН}$	$t_{ij}^{РО}$	$t_{ij}^{ПН}$	$t_{ij}^{ПО}$	R_{ij}	r_{ij}
A→B	4	0	4	4	8	4	4-4=0
A→C	6	0	6	0	6	0	0
B→D	3	4	7	8	11	4	0
C→E	5	6	11	6	11	0	0
D→F	4	7	11	15-4=11	15	4	15-11=4
E→F	4	11	15	11	15	0	0
F→G	3	15	18	15	18	0	0

max

max

min E

2. 1

$T_{кр}$ Критический путь

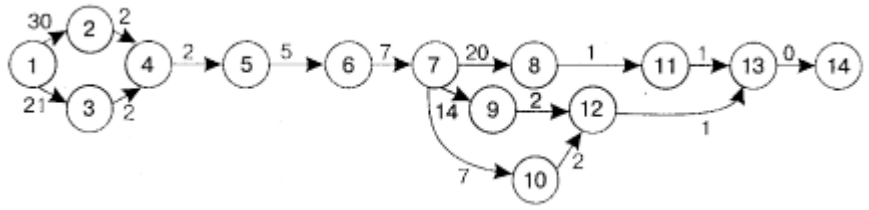
A
↓
C
↓
E
↓
F
↓
G

$T_{кр} = \Sigma t_{ij}$

$T_{кр} = 6+5+4+3 = 18$

Рисунок 1. Сетевой график

Пример №2:



Решение:

Работа	t_{ij}	РН t_{ij}	РО t_{ij}	МН t_{ij}	ПО t_{ij}	R_{ij}	r_{ij}	Кр. путь
1→2	30	0	30	0	30	0-0=0	30-30=0	1
1→3	21	0	21	9	30	9	0	2
2→4	2	30	30+2=32	30	32	0	0	4
3→4	2	21 max	23	30	32	9	9	5
4→5	2	32	34	32	34	0	0	6
5→6	5	34	39	34	39	0	0	7
6→7	7	39	46	39	46	0	0	8
7→8	20	46	66	46	66	0	0	11
7→9	14	46	60	51	65	0	0	13
7→10	7	46	53	58	65	0	0	14
8→11	1	66	67	66	67	0	0	
9→12	2	60	62	65	67	5	0	
10→12	2	53	55	65	67	12	0	
11→13	1	67	68	68-1=67	68	0	0	
12→13	1	62	63	67	68	5	0	
13→14	0	68	68	68	68	12	7	

min

$T_{кр} = 30 + 2 + 2 + 5 + 7 + 20 + 1 + 1 + 1 = \underline{68}$

Программная реализация

Алгоритм программы:

Структура Work — хранит данные о каждой работе:

Начальное и конечное событие (start, end)

Продолжительность работы (duration)

Раннее начало (t_early_start)

Раннее окончание (t_early_finish)

Позднее начало (t_late_start)

Позднее окончание (t_late_finish)

Полный резерв (full_reserve)

Частный резерв (reserve)

Класс Graph — основной вычислительный модуль:

1. **Загрузка данных:**
    a. Чтение графа из файла (формат: вершина, предшественник, вес)
    b. Автоматическая коррекция: если предшественник = 0, работа начинается с 1-й вершины
    c. Опциональное использование буквенного обозначения вершин (A-Z)
2. **Построение структуры графа:**
    a. Формирование списков смежности (adj_list) для прямого прохода
    b. Формирование списков предшественников (pred_list) для обратного прохода
3. **Расчет ранних сроков (прямой проход):**
    a. Инициализация ранних времен событий нулями
    b. Последовательный обход событий от 1 до N
    c. Для каждой работы: t_раннее_окончания = t_раннее_начала + продолжительность
    d. Определение максимального раннего времени для каждого события

4. **Расчет поздних сроков (обратный проход):**
    a. Определение завершающего события по максимальному раннему окончанию
    b. Инициализация позднего времени завершающего события
    c. Обратный обход от N к 1
    d. Для каждой работы: t_позднее_начала = t_позднее_окончания - продолжительность
5. **Вычисление резервов времени:**
    a. Полный резерв: R_ij = t_позднее_начала - t_раннее_начала
    b. Частный резерв: минимальное раннее начало последующих работ минус раннее окончание текущей
6. **Определение критического пути:**
    a. Поиск пути от начального до конечного события
    b. Критерий: работы с нулевыми полным и частным резервами
    c. Построение последовательности работ с проверкой посещенных вершин
7. **Вывод результатов:**
    a. Таблица со всеми рассчитанными параметрами в форматированном виде
    b. Шифр работ (буквенный или числовой)
    c. Продолжительность работ
    d. Ранние/поздние начала и окончания
    e. Полный и частный резервы
    f. Отметка критических работ
    g. Длина критического пути (максимальное раннее окончание)
    h. Последовательность работ критического пути
8. **Интерактивный режим:**
    a. Выбор использования тестовых данных (вариант 20)
    b. Выбор буквенного или числового обозначения вершин
    c. Возможность загрузки из пользовательского файла

Результат

Пример №1:

```
skif@DESKTOP-J0LT0TD:~/model/rep/model/lab2$ ./new.o
Использовать данные с моей карточки? (Вариант 20) (y/n): y
Использовать буквы английского алфавита для вершин? (y/n): y
Шифр    t(i,j)      t^PH_ij      t^PO_ij          t^ПН_ij      t^ПО_ij          R_ij          r_ij              Кр.
A-B         4           0            4               4           8              4           0
A-C         6           0            6               0           6              0           0            Критическая
B-D         3           4            7               8           11             4           0
C-E         5           6            11              6           11             0           0            Критическая
D-F         4           7            11              11          15             4           4
E-F         4           11           15              11          15             0           0            Критическая
F-G         3           15           18              15          18             0           0            Критическая
Длина критического пути: 18

Критический путь: A -> C -> E -> F -> G
```

Рисунок 2. Сетевой график (программная реализация)

Пример №2

```
skif@DESKTOP-J0LT0TD:~/model/rep/model/lab2$ ./new.o
Использовать данные с моей карточки? (Вариант 20) (y/n): n
Использовать буквы английского алфавита для вершин? (y/n): y
Введите имя файла с данными: test2.txt
Шифр    t(i,j)      t^PH_ij      t^PO_ij          t^ПН_ij      t^ПО_ij          R_ij          r_ij              Кр.
A-B         30          0            30              0           30             0           0            Критическая
A-C         21          0            21              9           30             9           0
B-D         2           30           32              30          32             0           0            Критическая
C-D         2           21           23              30          32             9           9
D-E         2           32           34              32          34             0           0            Критическая
E-F         5           34           39              34          39             0           0            Критическая
F-G         7           39           46              39          46             0           0            Критическая
G-H         20          46           66              46          66             0           0            Критическая
G-I         14          46           60              51          65             5           0
G-J         7           46           53              58          65             12          0
H-K         1           66           67              66          67             0           0            Критическая
I-L         2           60           62              65          67             5           0
J-L         2           53           55              65          67             12          7
K-M         1           67           68              67          68             0           0            Критическая
L-M         1           62           63              67          68             5           5
M-N         0           68           68              68          68             0           0            Критическая
Длина критического пути: 68

Критический путь: A -> B -> D -> E -> F -> G -> H -> K -> M -> N
```

:

Рисунок 3. Сетевой график (программаня реализация) №2

Вывод

В ходе выполнения данной работы была разработана программа для автоматизированного расчёта временных параметров сетевого графика. Программа реализует классический алгоритм метода критического пути, позволяющий эффективно решать задачи календарного планирования и управления проектами. Практическая значимость работы заключается в создании инструмента, который автоматизирует трудоёмкий процесс расчёта сетевых графиков. Программа позволяет не только получить точные значения всех временных параметров, но и визуально определить критические работы, требующие особого контроля при управлении проектом.

Приложение

main.cpp:

```
#include <algorithm>
#include <climits>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <queue>
#include <sstream>
#include <vector>

using namespace std;

struct Work {
   int start;
   int end;
   int duration;

   int t_early_start;
   int t_early_finish;
   int t_late_start;
   int t_late_finish;
   int full_reserve;
   int reserve;

   Work(int i, int j, int d)
      : start(i),
        end(j),
        duration(d),
        t_early_start(0),
        t_early_finish(0),
        t_late_start(0),
        t_late_finish(0),
        full_reserve(0),
        reserve(0)
   {
   }
};

class Graph {
private:
   int num_events;
   vector<Work> works;
   vector<vector<int>> adj_list;
   vector<vector<int>> pred_list;
   map<int, string> event_names;
   bool use_letters;

   string getEventName(int event_num) {
      if (use_letters && event_num >= 1 && event_num <= 26) {
```

```
        return string(1, 'A' + event_num - 1);
    }
    return to_string(event_num);
}

void calculateEarlyTimes()
{
    vector<int> early_time(num_events + 1, 0);

    for (int i = 1; i <= num_events; i++) {
        for (int work_idx : adj_list[i]) {
            Work& w = works[work_idx];
            int new_time = early_time[w.start] + w.duration;
            if (new_time > early_time[w.end]) {
                early_time[w.end] = new_time;
            }
        }
    }

    for (Work& w : works) {
        w.t_early_start = early_time[w.start];
        w.t_early_finish = w.t_early_start + w.duration;
    }
}

void calculateLateTimes()
{
    int max_event_time = 0;
    int last_event = 0;

    for (const Work& w : works) {
        if (w.t_early_finish >= max_event_time) {
            max_event_time = w.t_early_finish;
            last_event = w.end;
        }
    }

    vector<int> late_time(num_events + 1, INT_MAX);

    late_time[last_event] = max_event_time;

    for (int event = num_events; event >= 1; event--) {
        for (int work_idx : pred_list[event]) {
            Work& w = works[work_idx];

            if (late_time[w.end] != INT_MAX) {
                w.t_late_finish = late_time[w.end];

                w.t_late_start = w.t_late_finish - w.duration;

                if (late_time[w.start] > w.t_late_start) {
                    late_time[w.start] = w.t_late_start;
                }
```

```
            }
        }
    }

    for (Work& w : works) {
        if (w.t_late_finish == 0) {
            w.t_late_finish = late_time[w.end];
            w.t_late_start = w.t_late_finish - w.duration;
        }
    }
}

void calculateFloats()
{
    for (Work& w : works) {
        w.full_reserve = w.t_late_start - w.t_early_start;

        int min_early_start_next = INT_MAX;
        for (int next_idx : adj_list[w.end]) {
            const Work& next_work = works[next_idx];
            min_early_start_next
                = min(min_early_start_next, next_work.t_early_start);
        }

        if (min_early_start_next != INT_MAX) {
            w.reserve = min_early_start_next - w.t_early_finish;
        } else {
            w.reserve = 0;
        }
    }
}

vector<int> findCriticalPath()
{
    vector<int> path;
    vector<bool> visited(num_events + 1, false);

    int current = 1;
    path.push_back(current);
    visited[current] = true;

    while (current != num_events) {
        bool found = false;

        for (int work_idx : adj_list[current]) {
            Work& w = works[work_idx];

            if (w.full_reserve == 0 && w.reserve == 0 && w.start == current) {
                if (!visited[w.end]) {
                    current = w.end;
                    path.push_back(current);
                    visited[current] = true;
                    found = true;
```

```
                break;
            }
        }
    }

    if (!found) {
        break;
    }
  }

  return path;
}

public:
  Graph() : num_events(0), use_letters(false)
  {
  }

  void setUseLetters(bool use) {
    use_letters = use;
  }

  bool loadFromFile(const string& filename)
  {
    ifstream file(filename);
    if (!file.is_open()) {
        cerr << "ERROR: Не удалось открыть файл " << filename << endl;
        return false;
    }

    works.clear();
    adj_list.clear();
    pred_list.clear();
    event_names.clear();

    string line;
    int max_event = 0;
    vector<tuple<int, int, int>> temp_works;

    while (getline(file, line)) {
        if (line.empty())
            continue;

        istringstream iss(line);
        int ver, predecessor, weight;

        if (iss >> ver >> predecessor >> weight) {
            max_event = max(max_event, ver);
            if (predecessor > 0) {
                max_event = max(max_event, predecessor);
            }
            temp_works.push_back(make_tuple(predecessor, ver, weight));
        }
```

```
    }

    file.close();

    if (temp_works.empty()) {
        cerr << "ERROR: Файл не содержит данных" << endl;
        return false;
    }

    num_events = max_event;
    adj_list.resize(num_events + 1);
    pred_list.resize(num_events + 1);

    for (const auto& [pred, ver, weight] : temp_works) {
        if (pred > 0) {
            addWork(pred, ver, weight);
        } else {
            addWork(1, ver, weight);
        }
    }

    return true;
}

void addWork(int i, int j, int duration)
{
    works.push_back(Work(i, j, duration));
    int index = works.size() - 1;
    adj_list[i].push_back(index);
    pred_list[j].push_back(index);
}

void calculateAll()
{
    if (works.empty()) {
        cerr << "ERROR: Нет данных для расчета" << endl;
        return;
    }
    calculateEarlyTimes();
    calculateLateTimes();
    calculateFloats();
}

void printTable()
{
    if (works.empty()) {
        cout << "Нет данных для отображения" << endl;
        return;
    }

    cout << left << setw(12) << "Шифр" << setw(12) << "t(i,j)" << setw(15)
         << "t^РН_ij" << setw(19) << "t^РО_ij" << setw(15) << "t^ПН_ij"
         << setw(19) << "t^ПО_ij" << setw(14) << "R_ij" << setw(18)
```

```
            << "r_ij"
            << "Кр." << endl;

        int critical_path_length = 0;
        for (const Work& w : works) {
            critical_path_length = max(critical_path_length, w.t_early_finish);
        }

        for (size_t k = 0; k < works.size(); k++) {
            const Work& w = works[k];

            bool is_critical = (w.full_reserve == 0 && w.reserve == 0);

            string code = getEventName(w.start) + '-' + getEventName(w.end);

            cout << left << setw(12) << code << setw(12) << w.duration
                << setw(15) << w.t_early_start << setw(15) << w.t_early_finish
                << setw(11) << w.t_late_start << setw(15) << w.t_late_finish
                << setw(12) << w.full_reserve << setw(12) << w.reserve;

            if (is_critical)
                cout << " Критическая";

            cout << endl;
        }

        cout << "Длина критического пути: " << critical_path_length << endl;

        cout << "\nКритический путь: ";
        vector<int> critical_path = findCriticalPath();

        for (size_t i = 0; i < critical_path.size(); i++) {
            cout << getEventName(critical_path[i]);
            if (i < critical_path.size() - 1)
                cout << " -> ";
        }
        cout << endl;

    }

    void findAndPrintCriticalPath()
    {
        vector<int> path = findCriticalPath();

        for (size_t i = 0; i < path.size(); i++) {
            cout << getEventName(path[i]);
            if (i < path.size() - 1)
                cout << " -> ";
        }
    }
};

int main()
```

```
{
    Graph graph;
    char choice;
    char letter_choice;

    cout << "Использовать данные с моей карточки? (Вариант 20) (y/n): ";
    cin >> choice;

    cout << "Использовать буквы английского алфавита для вершин? (y/n): ";
    cin >> letter_choice;

    graph.setUseLetters(letter_choice == 'y' || letter_choice == 'Y');

    if (choice == 'y' || choice == 'Y') {
        Graph test_graph;
        test_graph.setUseLetters(letter_choice == 'y' || letter_choice == 'Y');

        ofstream test_file("test_graph.txt");
        test_file << "2 1 4\n";
        test_file << "3 1 6\n";
        test_file << "4 2 3\n";
        test_file << "5 3 5\n";
        test_file << "6 4 4\n";
        test_file << "6 5 4\n";
        test_file << "7 6 3\n";
        test_file.close();

        if (test_graph.loadFromFile("test_graph.txt")) {
            test_graph.calculateAll();
            test_graph.printTable();
        }
    } else {
        string filename;

        cout << "Введите имя файла с данными: ";
        cin >> filename;

        if (graph.loadFromFile(filename)) {
            graph.calculateAll();
            graph.printTable();
        }
    }

    return 0;
}
```